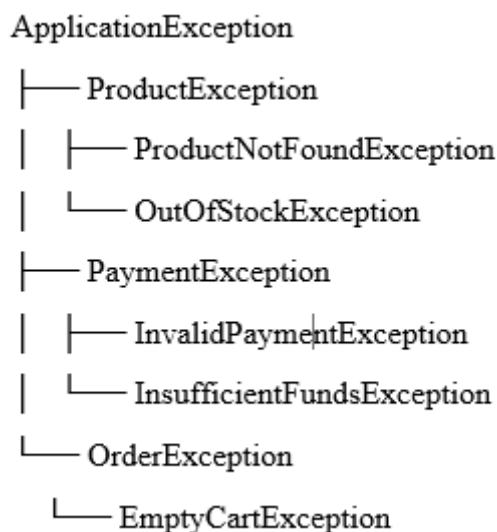


List of Experiments (Exception & Debugging)
EWA(UCS553)
(Week 5)

1. Create a program that demonstrates the hierarchy of common exceptions such as `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException`.
2. Write a program containing nested try-catch blocks. Generate different exceptions in the inner and outer blocks and demonstrate how exception propagation works.
3. Develop a calculator that accepts two numbers and an operator (+, -, *, /). It should be able to handle the following exceptions: Division by zero, Invalid numeric input, Invalid operators. Use separate exception handlers for different errors.
4. Create a student result-processing program that accepts marks for several subjects. Marks must be between 0 and 100. Create a custom `InvalidMarksException`. Throw the exception when invalid marks are entered. Calculate total, percentage, and grade only when all input is valid. Display a meaningful error message.
5. Implement a simple banking application supporting deposit and withdrawal. Create a custom `InsufficientBalanceException`. Create a custom `InvalidAmountException`. Prevent withdrawal when the balance is insufficient. Prevent zero or negative deposits/withdrawals. Demonstrate exception propagation from one method to another. Display the account balance after successful transactions. Add `AccountNotFoundException` for invalid account numbers.
6. Develop a login system with exception-based validation. Create custom exceptions such as: `InvalidUsernameException`, `InvalidPasswordException`, `AccountLockedException`. Allow only a fixed number of failed login attempts. Lock the account after the maximum attempts. Use try-catch-finally appropriately. Display different messages for different failures.
7. Design a small shopping/cart application that uses a complete exception hierarchy. Given below is the suggested hierarchy



Implement the appropriate operations like add/remove, payment, search products etc. Use appropriate exception handlers and debug the program using breakpoints and variable inspection.

